

Problem 3 (Hujinator)

Geva Haviv

Part 1: The Quest of the Hujinator

1. Initial Entropy

כדי לחשב את האנטרופיה ההתחלתית של מאגר הנתונים, נסתכל על עמודת המטרה (Beloved). מתוך $N = 10$ מרצים, ישנם 6 "כן" (Y) ו-4 "לא" (N).

ההסתברויות הן:

$$P(Y) = \frac{6}{10} = 0.6 \quad \square$$

$$P(N) = \frac{4}{10} = 0.4 \quad \square$$

נוסחת האנטרופיה היא:

$$H(S) = -P(Y) \log_2(P(Y)) - P(N) \log_2(P(N))$$

נציב את הערכים:

$$H(S) = -0.6 \log_2(0.6) - 0.4 \log_2(0.4)$$

$$H(S) = -0.6(-0.737) - 0.4(-1.322)$$

$$H(S) \approx 0.971$$

2. Information Gain (IG)

א. $Gain(S, A)$: מלמד מדעי המחשב (Teaches CS)

□ עבור $A = 1$: סה"כ 5 (3 Y, 2 N). האנטרופיה:

$$H(S_{A=1}) = -\frac{3}{5} \log_2\left(\frac{3}{5}\right) - \frac{2}{5} \log_2\left(\frac{2}{5}\right) \approx 0.971$$

□ עבור $A = 0$: סה"כ 5 ($N = 2, Y = 3$). האנטרופיה:

$$H(S_{A=0}) = -\frac{3}{5} \log_2 \left(\frac{3}{5} \right) - \frac{2}{5} \log_2 \left(\frac{2}{5} \right) \approx 0.971$$

□ רווח המידע:

$$Gain(S, A) = 0.971 - \left(\frac{5}{10} \cdot 0.971 + \frac{5}{10} \cdot 0.971 \right) = 0$$

ב. $Gain(S, B)$: נותן בונוס (Gives Bonuses)

□ עבור $B = 1$: סה"כ 6 ($N = 0, Y = 6$). האנטרופיה $H(S_{B=1}) = 0$ (צומת טהור).

□ עבור $B = 0$: סה"כ 4 ($N = 4, Y = 0$). האנטרופיה $H(S_{B=0}) = 0$ (צומת טהור).

□ רווח המידע:

$$Gain(S, B) = 0.971 - \left(\frac{6}{10} \cdot 0 + \frac{4}{10} \cdot 0 \right) = 0.971$$

ג. $Gain(S, C)$: מרכיב משקפיים (Wears Glasses)

□ עבור $C = 1$: סה"כ 6 ($N = 1, Y = 5$). האנטרופיה:

$$H(S_{C=1}) = -\frac{5}{6} \log_2 \left(\frac{5}{6} \right) - \frac{1}{6} \log_2 \left(\frac{1}{6} \right) \approx 0.650$$

□ עבור $C = 0$: סה"כ 4 ($N = 3, Y = 1$). האנטרופיה:

$$H(S_{C=0}) = -\frac{1}{4} \log_2 \left(\frac{1}{4} \right) - \frac{3}{4} \log_2 \left(\frac{3}{4} \right) \approx 0.811$$

□ רווח המידע:

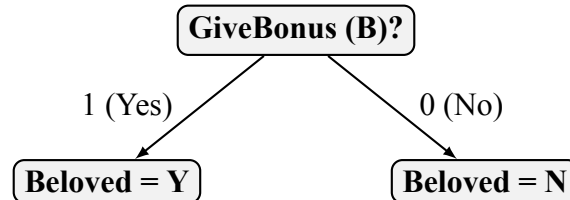
$$Gain(S, C) = 0.971 - \left(\frac{6}{10} \cdot 0.650 + \frac{4}{10} \cdot 0.811 \right) = 0.971 - 0.714 = 0.257$$

הסבר התוצאות:

התכונה B מניבה את רווח המידע המקסימלי האפשרי (0.971) מכיוון שהיא מפצלת את הנתונים באופן מושלם לתת-קבוצות טהורות עם אנטרופיה אפס. התכונה A מניבה רווח מידע של 0 כיוון שהתפלגות משתנה המטרה בתת-הקבוצות שלה זהה לחלוטין להתפלגות במאגר הנתונים המקורי. התכונה C מניבה רווח מידע מתון (0.257), כלומר היא מפחיתה מעט את חוסר הוודאות, אך אינה מפרידה את המחלקות באופן מושלם.

3. Tree Construction

בהתבסס על החישובים בשאלה 2, ה-Hujinator יבחר בתכונה **GiveBonus (B)** כשורש העץ (Root Node) מכיוון שיש לה את רווח המידע הגבוה ביותר (0.971). מאחר שתכונה זו מפצלת את הנתונים באופן מושלם לעלים טהורים (אנטרופיה = 0), העץ אינו דורש פיצולים נוספים.



4. Banning Bonuses

אם תכונה B מוסרת, ה-Hujinator צריך לבחור בין תכונה A לתכונה C כשורש העץ החדש. במקרה זה, הוא יבחר בתכונה **HasGlasses (C)**.

הצדקה:

אלגוריתמים של עצי החלטה בוחרים בכל שלב את התכונה שמספקת את רווח המידע (Information Gain) הגבוה ביותר, כדי למקסם את הפחתת חוסר הודאות. על פי החישובים הקודמים שלנו:

$$Gain(S, C) = 0.257 \quad \square$$

$$Gain(S, A) = 0 \quad \square$$

מכיוון ש- $0.257 > 0$, פיצול הנתונים לפי הרכבת משקפיים (C) מספק הרבה יותר ערך ניבוי מאשר פיצול לפי קריטריון מדעי המחשב (A), שלא מספק שום ערך ניבוי כלל.

חלק 2:

דו"ח באגים

להלן רשימת הבאגים שזוהו בקוד `hujinator.py` והתיקונים שבוצעו:

1. באג #1 - נוסחת האנטרופיה הפוכה וב-base שגוי

□ **Location:** שורה 18, פונקציה `calculate_entropy()`
□ **Diagnostic:** אנטרופיית שאנון מוגדרת כ- $H = -\sum p \cdot \log_2(p)$. הקוד המקורי חישב $+\sum p \cdot \ln(p)$ - ללא סימן מינוס ותוך שימוש בלוגריתם טבעי במקום בסיס 2. התוצאה היא ערך שלילי במקום חיובי.
□ **Correction:** שינוי ל-`if p > 0: -sum(p * np.log2(p) for p in probabilities)`

2. באג #2 - משקל קבוע במקום יחסי

□ **Location:** שורה 29, פונקציה `calculate_information_gain()`
□ **Diagnostic:** המשקל של כל תת-קבוצה צריך להיות $\frac{|S_v|}{|S|}$. במקום זאת, הוכנס ערך קבוע `weight = 2`, מה שהופך את החישוב לחסר משמעות סטטיסטית.
□ **Correction:** שינוי ל-`weight = len(subset) / len(data)`

3. באג #3 - נוסחת Information Gain הפוכה

□ **Location:** שורה 32, פונקציה `calculate_information_gain()`
□ **Diagnostic:** $IG = H(S) - \sum \frac{|S_v|}{|S|} H(S_v)$. הקוד חישב `weighted - total` - כלומר ההפך: ערך שלילי במקום חיובי.
□ **Correction:** שינוי ל-`return total_entropy - weighted_entropy`

4. באג #4 - ערכי ברירת מחדל מסוג Ellipsis

□ **Location:** שורה 35, חתימת הפונקציה `build_tree()`
□ **Diagnostic:** הפרמטרים `max_depth=...` ו-`min_samples_split=...` מקבלים את האובייקט `Ellipsis` של Python. השוואות מספריות מולם נכשלות או מחזירות תוצאות שגויות.
□ **Correction:** קביעת ערכי ברירת מחדל סבירים: `min_samples_split=2, max_depth=10`

5. באג #5 - חסר תנאי עצירה ל-max_depth

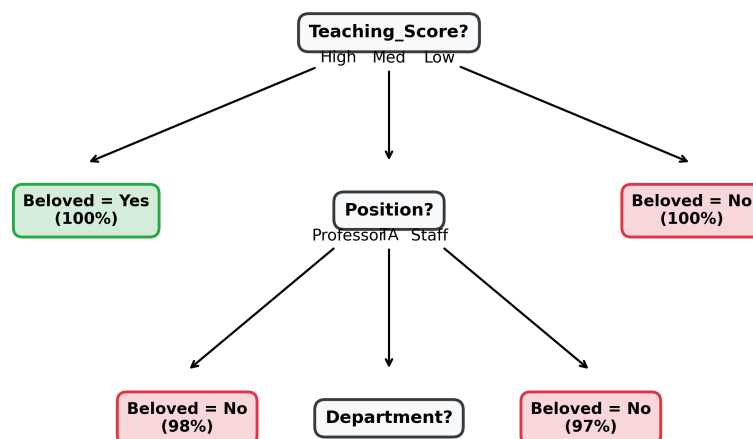
□ **Location:** שורות 39-53, פונקציה `build_tree()`
□ **Diagnostic:** לא קיימת בדיקה האם `depth >= max_depth`. העץ יכול לגדול ללא הגבלה ולגרום ל-`overfitting`.
□ **Correction:** הוספת תנאי: `if not features or depth >= max_depth:` - החזרת עלה עם מחלקת הרוב.

6. באג #6 - סף Information Gain אבסורדי (200)

- **Location:** שורה 52, פונקציה `build_tree()`
- **Diagnostic:** IG תמיד בטווח $[0, \log_2(k)]$ (עבור סיווג בינארי: עד 1.0). סף של 200 גורם לכך שכל פיצול נדחה והעץ מתנוון לעלה בודד.
- **Correction:** שינוי ל-: `if gains[best_feature] <= 0`.
- 7. באג #7 - שימוש ב-`subset.size` במקום `len(subset)`
- **Location:** שורה 66, פונקציה `build_tree()`
- **Diagnostic:** `DataFrame.size` מחזיר `rows × columns` (למשל, 7 עמודות × שורה אחת = 7), ולא את מספר הדגימות. זה גורם לתת-קבוצות קטנות לעבור את הסף שלא בצדק.
- **Correction:** שינוי ל-: `if len(subset) < min_samples_split`.

Task B: Visual Evidence

לאחר תיקון כל הבאגים, הופעל הסקריפט המתוקן והופקה ויזואליזציה של עץ ההחלטה הסופי.



איור 1: עץ ההחלטה הסופי של ה-Hujinator לאחר תיקון הבאגים.

Task B: Predictor Logic

ניתוח לוגיקת עץ ההחלטה - ה-Hujinator

עץ ההחלטה שנבנה באמצעות אלגוריתם ID3 חושף את המאפיינים המנבאים ביותר האם מרצה באוניברסיטה העברית ייחשב כ-*Beloved* (אהוב) על ידי הסטודנטים.

צומת השורש: Teaching_Score

המאפיין בעל ה-Information Gain הגבוה ביותר ($IG = 0.703$) הוא Teaching_Score. המשמעות ברורה: **איכות ההוראה היא הגורם הדומיננטי ביותר** בקביעה האם מרצה יהיה אהוב.

□ Teaching_Score = High $\Rightarrow$ Beloved = Yes (347 דגימות, 100% דיוק).
מרצה עם ציון הוראה גבוה **תמיד** נחשב אהוב – ללא תלות בתפקיד, מחלקה, או ותק.

□ Teaching_Score = Low $\Rightarrow$ Beloved = No (305 דגימות, 100% דיוק).
מרצה עם ציון הוראה נמוך **לעולם** לא נחשב אהוב – ללא קשר לפרמטרים אחרים.

□ Teaching_Score = Med $\Rightarrow$ (348 דגימות, Yes 26% / No 74%).
נדרש פיצול נוסף

רמה שנייה (עבור Med): Position

כאשר ציון ההוראה בינוני, הגורם המכריע הבא ($IG = 0.348$) הוא **התפקיד**:

□ Position = Professor $\Rightarrow$ Beloved = No (110 דגימות, 98% דיוק).
פרופסורים עם ציון בינוני כמעט תמיד אינם אהובים.

□ Position = Staff $\Rightarrow$ Beloved = No (102 דגימות, 97% דיוק).
אנשי סגל עם ציון בינוני גם כן אינם אהובים.

□ Position = TA $\Rightarrow$ (136 דגימות, No 38% / Yes 62%).
עבור מתרגלים עם ציון בינוני, הגורם הנוסף הוא Department ($IG = 0.317$).

מסקנות מרכזיות

1. **ציון ההוראה הוא הגורם המכריע** – הוא לבדו מסווג נכון 65% מהנתונים (High $\rightarrow$ Yes, Low $\rightarrow$ No).

2. **מתרגלים נהנים מיתרון** – בציון בינוני, רק מתרגלים (TAs) יש להם סיכוי להיחשב אהובים, בעוד פרופסורים ואנשי סגל נדחים.

3. **סוג המשרד (Office_Type)** אינו רלוונטי – $IG = 0$; אין לו שום ערך מנבא.

4. **מחלקה (Department)** רלוונטית רק עבור TAs בינוניים – במקרים אחרים השפעתה זניחה.

לסיכום, ה-Hujinator מלמד אותנו כי **איכות ההוראה גוברת על הכל**: מרצה מצוין יהיה אהוב תמיד, מרצה גרוע לא יהיה אהוב לעולם, ובמקרי הביניים – מתרגלים צעירים במחלקות מסוימות עדיין יכולים לזכות בהערכת הסטודנטים.